

Design Quality Analysis Report

Team Name- ASM

Team Number - 16

Members-

Arnav Agnihotri (2024101103)

Samyak Jain (2024101062)

Manik Bansal (2024101084)

5.1 Redundancy Analysis

Identified Redundancies:

1. Warrior Name Redundancy

Location: The Name attribute in the Warrior entity is shown as a multivalued attribute (pink/red oval)

Problem: Storing multiple names for the same warrior (e.g., birth name, title-based name, aliases) can lead to:

- **Data inconsistency:** If a warrior's name needs to be updated, all instances must be changed
- **Storage waste:** Same information repeated multiple times
- **Query complexity:** Searching for a warrior becomes complicated when multiple name variations exist

Example: Arjuna might be stored as "Arjuna", "Partha", "Dhananjaya" - if we need to update his primary name, we must ensure all variations are correctly maintained

2. Classification Information Redundancy

Location: Warrior entity has a Classification attribute, but classification is also implied through the ISA hierarchy (Maharathi, Atirathi, Rathi)

Problem: The same classification information exists in two places:

- As a direct attribute of Warrior
- As membership in a specialized entity type

Issues this causes:

- **Update anomaly:** If a warrior's classification changes (e.g., from Rathi to Atirathi after gaining skills), we must update both the Classification attribute AND move the warrior to a different entity subtype
- **Inconsistency risk:** The Classification attribute might say "Maharathi" while the warrior exists in the "Rathi" entity, creating contradictory data

3. Kingdom Allegiance Redundancy

Location: Kingdom entity has an Allegiance attribute, and there's also a Pledges_allegiance relationship between Kingdom and Faction

Problem: Kingdom's faction allegiance is stored twice:

- As an attribute within Kingdom
- As a relationship to Faction entity

Issues:

- **Data inconsistency:** The Allegiance attribute might say "Kaurava" while the relationship points to "Pandava" faction
- **Maintenance overhead:** Any change in allegiance requires updates in two places
- **Redundant storage:** The same information (which faction a kingdom belongs to) is duplicated

4. Victor Faction in Battle

Location: Battle entity has Victor_faction attribute

Problem: If this stores the faction name rather than just a reference, the faction's name is duplicated

Issues:

- If Faction name changes, all historical Battle records must be updated
- This is actually acceptable as a historical record (denormalization for query performance), but could cause inconsistency if not properly managed

5.2 Update Anomaly

Yes, update anomalies exist. Here are specific examples:

Example 1: Warrior Classification Update

Scenario: A Rathi warrior (like Abhimanyu initially) gains sufficient skills and experience to become an Atirathi

Update required in multiple places:

- Update the Classification attribute in the Warrior entity from "Rathi" to "Atirathi"
- Remove the warrior record from the Rathi subtype entity
- Create a new record in the Atirathi subtype entity with the attribute Chariot_combat_rating
- Transfer all relationships (Assigned_to chariot, Possesses astras, etc.) to the new subtype record

Problem: If any of these updates fail or are forgotten, the database enters an inconsistent state where a warrior's classification doesn't match their actual subtype entity

Example 2: Kingdom Allegiance Change

Scenario: A kingdom switches sides during the war (like some kingdoms historically did before the Kurukshetra war)

Update required in multiple places:

- Update the Allegiance attribute in the Kingdom entity
- Delete the old Pledges_allegiance relationship to the previous Faction
- Create a new Pledges_allegiance relationship to the new Faction

Problem: Without proper transaction management, the kingdom could temporarily appear to have no allegiance or wrong allegiance, affecting all dependent queries and reports

Example 3: Faction Name Update

Scenario: A faction's name needs to be corrected or updated (e.g., "Pandava" to "Pandava Alliance")

Update required in multiple places:

- Update the Name attribute in the Faction entity
- Update all Victor_faction attributes in Battle records that reference this faction
- Update the Allegiance attribute in all Kingdom records that reference this faction name

Problem: Missing any of these updates creates inconsistency where the same faction is referred to by different names across the database

Example 4: Army Unit Composite Key Update When Kingdom Changes

Scenario: An army unit needs to be transferred from one kingdom to another (e.g., during political realignment)

Update required:

- Since Army_unit is a weak entity with composite key "Full key(Kingdom id + unit id)", changing the kingdom changes part of the primary key
- All foreign key references to this army unit in other tables must be updated
- The Bestowed_with relationship between Boon_curse and Army_unit must be updated
- Any historical records referencing the old composite key become invalid

Problem: Updating a primary key component is extremely problematic and can break referential integrity across the entire database.

5.3 Insertion Anomaly

Yes, insertion anomalies exist:

Scenario 1: Cannot Insert Army_unit Without a Kingdom

Situation: Army_unit is a weak entity dependent on Kingdom, with a composite key "Full key(Kingdom id + unit id)"

Problem: You cannot create an army unit unless a kingdom already exists in the database

Real-world issue: If you want to plan generic army unit structures (templates) before assigning them to specific kingdoms, you cannot do so. For example:

- Planning to create a standard "Infantry Division" template with specific Type, Initial_strength, and Description
- This unit template cannot exist in the database until a kingdom is assigned
- Forces premature decision-making or requires creating dummy kingdom records

Example: Cannot insert "Elite Cavalry Unit" with Initial_strength = 5000 until you first decide whether it belongs to Hastinapur, Panchal, or another kingdom

Scenario 2: Cannot Insert Boon_curse Without Kingdom and Army_unit

Situation: Boon_curse is a weak entity with composite key "warrior id + boon_curse name", and it has a Bestowed_with relationship to Army_unit (which itself depends on Kingdom)

Problem: You cannot record that a boon or curse exists unless:

- A kingdom already exists
- An army unit from that kingdom already exists
- A warrior already exists to associate with the boon/curse

Real-world issue: If you're cataloging all known boons/curses from mythology before assigning them to specific contexts, you cannot store this information independently

Example: You cannot insert "Karna's curse that he will forget Brahmastra mantra at critical moment" until:

- A kingdom exists (e.g., Anga)
- An army unit from that kingdom exists
- The boon/curse is associated with that army unit through Bestowed_with

Scenario 3: Cannot Insert Kingdom Without Faction

Situation: The Pledges_allegiance relationship has cardinality (1,1) from Kingdom to Faction

Problem: You cannot insert a new kingdom unless a faction already exists for it to pledge allegiance to

Real-world issue:

- During the early stages of conflict, kingdoms existed before formal faction alliances were made
- If you want to record the kingdom of "Matsya" before it decided whether to support Pandavas or Kauravas, you cannot

- Neutral or undecided kingdoms cannot be recorded
- This forces you to either create a "Neutral" faction (workaround) or delay recording kingdom information

Example: Cannot insert kingdom "Virata" with its Capital_city "Upaplavya" until either Pandava or Kaurava faction exists

Scenario 4: Cannot Insert Warrior Without Kingdom

Situation: The Belongs_to relationship has cardinality (0,N) from Kingdom to Warrior, meaning a warrior must belong to a kingdom

Problem: You cannot insert warrior information for:

- Wandering warriors or hermits not affiliated with any kingdom
- Warriors whose kingdom affiliation is unknown or disputed
- Legendary warriors you want to catalog before determining their kingdom

Example: Cannot insert "Barbarika" (Krishna's devotee) if he's not affiliated with any kingdom, even though he's an important character in the mythology

5.4 Deletion Anomaly

Yes, deletion anomalies exist:

Example 1: Deleting the Only Warrior in a Classification Type

Situation: If you delete the last remaining Maharathi warrior from the database

Unintended loss:

- You lose the entire Maharathi entity definition and its specific attribute Master_of_all_astras
- The classification category itself disappears from the database structure
- Any business rules or application logic specifically handling Maharathi warriors may break

Problem: Deleting data (warrior instance) causes loss of metadata (the classification category and its schema)

Example 2: Deleting a Kingdom Cascades to Army Units and Warriors

Situation: Delete a kingdom like "Matsya" or "Panchal"

Unintended loss: Since Army_unit is a weak entity dependent on Kingdom:

- All army units from that kingdom are automatically deleted, including:
 - Their Type information (Infantry, Cavalry, Elephant corps, etc.)
 - Their Initial_strength data
 - Their Description
 - Their deployment history

- All Boon_curse records associated with those army units through Bestowed_with relationship are also deleted
- All warriors from that kingdom become orphaned or must be deleted (depending on referential integrity settings)
 - Warriors' battle participation history is lost
 - Warriors' Astra possessions are lost
 - Warriors' chariot assignments are lost

Cascading data loss chain:

Delete Kingdom → Deletes all Army_units → Deletes all Boon_curse records → Orphans/deletes all Warriors → Loses all warrior-related information

Example: Deleting "Virata" kingdom would delete:

- All Matsya army units (infantry, cavalry divisions)
- All boons/curses associated with Matsya forces
- Information about Uttara, Shankha, and all other Matsya warriors
- Historical records of their participation in the war

Example 3: Deleting a Faction Orphans Multiple Kingdoms

Situation: Delete the "Pandava" or "Kaurava" faction

Unintended loss: Since Pledges_allegiance has cardinality (1,1) from Kingdom side:

- All kingdoms that pledged allegiance to that faction become invalid (cannot exist without a faction)
- This cascades massively:
 - Kingdoms must be deleted or become invalid
 - All army units from those kingdoms are deleted (weak entity dependency)
 - All boon/curse records associated with those army units are deleted
 - All warriors from those kingdoms are orphaned/deleted

Catastrophic data loss chain:

Delete Faction → Deletes/Orphans Kingdoms → Deletes Army_units → Deletes Boon_curse records → Deletes Warriors

Entire war sides could be erased by deleting one faction entity

Example: Deleting "Pandava" faction would potentially delete:

- Kingdoms: Indraprastha, Panchal, Matsya, Kasi, Chedi
- All army units from these kingdoms

- All warriors including Yudhishtira, Bhima, Arjuna, Nakula, Sahadeva, Dhrishtadyumna, Shikhandi, etc.
- All historical battle records involving these entities

Example 4: Deleting an Army Unit Loses Boon/Curse Information

Situation: Delete an army unit like "Bhishma's Northern Guard" from Hastinapur

Unintended loss:

- All boon/curse records associated with that specific army unit through Bestowed_with relationship are deleted
- Historical information about special powers or curses affecting that unit is permanently lost
- If this was the only army unit with certain boons, that knowledge is completely erased

Problem: You might want to disband an army unit (organizational change) without losing the historical record of what boons or curses were associated with it

Example: If you delete "Drona's Archer Battalion", you lose records of:

- "Blessing of Unerring Aim" bestowed upon the unit
- "Curse of Friendly Fire" that affected the unit
- Any special divine protections or vulnerabilities

Example 5: Deleting the Last Battle Record Loses Victor Information

Situation: If there's only one battle where a particular faction won, and you delete that battle record

Unintended loss:

- The only proof that the faction achieved victory is lost
- If Victor_faction is not stored redundantly in the Faction entity, there's no way to know this faction ever won any battles
- Historical analysis of faction performance becomes impossible

Note: This is acceptable in proper normalization (battles are events that can be deleted), but it illustrates how deletion removes historical information that might be important for analysis

Example 6: Cannot Delete a Kingdom With Army Units

Situation: Attempting to delete a kingdom that has army units

Problem: This is actually a blocked deletion rather than unintended loss, but it's still an anomaly:

- You cannot delete a kingdom (even if you want to for data cleanup) if any army units depend on it
- Forces you to first manually delete all army units, which cascades to boon/curse records
- Makes legitimate administrative operations unnecessarily complex

Example: Cannot remove "Sindhu" kingdom from the database even if it's duplicate data, unless you first delete all its army units

Summary

The ER diagram has several significant data quality issues that need addressing:

Redundancies exist in multiple places:

- Multivalued warrior names causing storage and consistency issues
- Duplicate classification information (attribute vs. ISA hierarchy)
- Kingdom allegiance stored both as attribute and relationship
- Potential faction name duplication in battle records

Update anomalies require multi-location updates for:

- Warrior classification changes (attribute + entity migration)
- Kingdom allegiance switches (attribute + relationship)
- Faction name modifications (propagating through multiple tables)
- Army unit kingdom transfers (composite primary key changes)

Insertion anomalies prevent creating:

- Army units without kingdoms (weak entity dependency)
- Boon/curse records without both warriors and army units
- Kingdoms without factions (mandatory relationship)
- Warriors without kingdom affiliation
- Generic templates or unaffiliated entities

Deletion anomalies cause cascading data loss, particularly severe:

- Deleting Kingdom → cascades to Army_units → Boon_curse records → Warriors (massive data loss)
- Deleting Faction → cascades through Kingdoms → Army_units → Boon_curse → Warriors (catastrophic loss of entire war sides)
- Deleting Army_unit → loses all associated boon/curse historical records
- Blocked deletions prevent legitimate administrative operations due to weak entity dependencies

The most critical issue is the weak entity dependency of Army_unit on Kingdom, which creates a brittle structure where deleting a single kingdom can cascade through multiple levels, potentially losing vast amounts of historical and organizational data. This should be reconsidered based on whether army units should be able to exist independently or be transferred between kingdoms.

ER DIAGRAM

